4-Bit Array Multiplier from RTL to GDSII with Open-Source Tools

Digital VLSI Design (M26EC2.408) Assignment Report (Team Madhurbala_riders)

Vansh Jaiswal
ECE, IIIT Hyderabad
2024102013
vansh.jaiswal@students.iiit.ac.in

Saarthak Sabharwal
ECE, IIIT Hyderabad
2023102055
saarthak.sabharwal@students.iiit.ac.in

***Abstract*—This report presents the design, verification and complete open-source RTL-to-GDSII implementation of a registered 4-bit $\times$ 4-bit unsigned array multiplier in the SkyWater SKY130 130 nm process with the `sky130_fd_sc_hd` standard-cell library. The multiplier, built from 16 AND gates and 12 full adders between an input and an output register, is verified exhaustively in RTL and gate-level simulation (256/256 vectors). Yosys maps it to 63 cells (740.71 μm^2, flip-flops 54.05 % of the area); OpenSTA gives a minimum period of 2.604 ns (384.0 MHz) at the typical corner, with a critical path through 7 full-adder stages. LibreLane then places and routes the design on a 150 $\times$ 150 μm die with 0 DRC and 0 LVS errors, closing multi-corner setup timing at a 6.5 ns clock (post-route F_{max} 155.9 MHz). The gap between the pre-layout and post-route frequency is analysed in terms of process corners, interconnect, the clock tree and the cells added by the physical-design flow.**

I. Introduction

Multiplication is a core operation in arithmetic and signal-processing datapaths. The array multiplier is the most regular hardware realisation of shift-and-add multiplication: every partial product is formed by a row of AND gates and the rows are summed by ripple-carry adders laid out as a two-dimensional array. Its regularity makes it a good vehicle for exercising a complete design flow, while its carry chains give a clearly identifiable critical path whose length grows linearly with the word length.

In this assignment a registered 4-bit array multiplier is taken through the full open-source flow:

- RTL design and exhaustive self-checking simulation (Icarus Verilog, GTKWave),
- logic synthesis to SKY130 standard cells (Yosys) and gate-level simulation,
- static timing analysis (OpenSTA),
- floorplanning, placement, clock-tree synthesis, routing and signoff (LibreLane with OpenROAD, Magic, KLayout and Netgen), producing the final GDSII layout.

All tools were run from the IIC-OSIC-TOOLS Docker image [3]. The exact command for every step is listed in the repository `README.md`.

II. Theory and Proposed Design

A. Partial Products

For two 4-bit unsigned operands $A = a_3a_2a_1a_0$ and $B = b_3b_2b_1b_0$, multiplication is shift-and-add [1], [2]. Each bit b_i of the multiplier selects one partial product

$$pp_i[j] = a_j \wedge b_i, \qquad i, j \in \{0, \ldots, 3\}, \tag{1}$$

which is weighted by 2^i, so that

$$P = \sum_{i=0}^{3} 2^i \sum_{j=0}^{3} 2^j \, pp_i[j], \qquad 0 \le P \le 225. \tag{2}$$

B. Array Structure

The array adds the partial products one row at a time:

1) Row 0 consists only of AND gates. Its lowest bit is the final product bit $p_0 = pp_0[0]$ and the other three bits are passed down as $r_0 = \{0,\, pp_0[3],\, pp_0[2],\, pp_0[1]\}$.
2) Rows $i = 1, 2, 3$ are 4-bit ripple-carry adders. Full adder j of row i adds $pp_i[j]$, $r_{i-1}[j]$ and the carry $c_i[j]$ (with $c_i[0] = 0$), producing the sum $s_i[j]$ and the carry $c_i[j+1]$.
3) The lowest sum bit of each row is a finished product bit, $p_i = s_i[0]$, and the row passes down $r_i = \{c_i[4],\, s_i[3],\, s_i[2],\, s_i[1]\}$.
4) The last row also gives the top four product bits, $p_{7:4} = r_3$.

Each full adder computes

$$s = a \oplus b \oplus c_{in}, \tag{3}$$

$$c_{out} = ab + a\,c_{in} + b\,c_{in}. \tag{4}$$

In general an N-bit array multiplier needs N^2 AND gates and $N(N-1)$ full adders, i.e. 16 AND gates and 12 full adders for $N = 4$, and its longest path is roughly $2N$ full-adder delays.

C. Registers and Latency

The operands are registered into `a_r` and `b_r` on every rising clock edge, the array is built from the registered values, and the 8-bit result is registered into `p`. This gives exactly 16 flip-flops, all with an asynchronous active-low reset `rst_n`. An input applied before clock edge 1 is captured at edge 1 and the product appears at edge 2: the latency is 2 clock cycles and the throughput is one product per cycle.

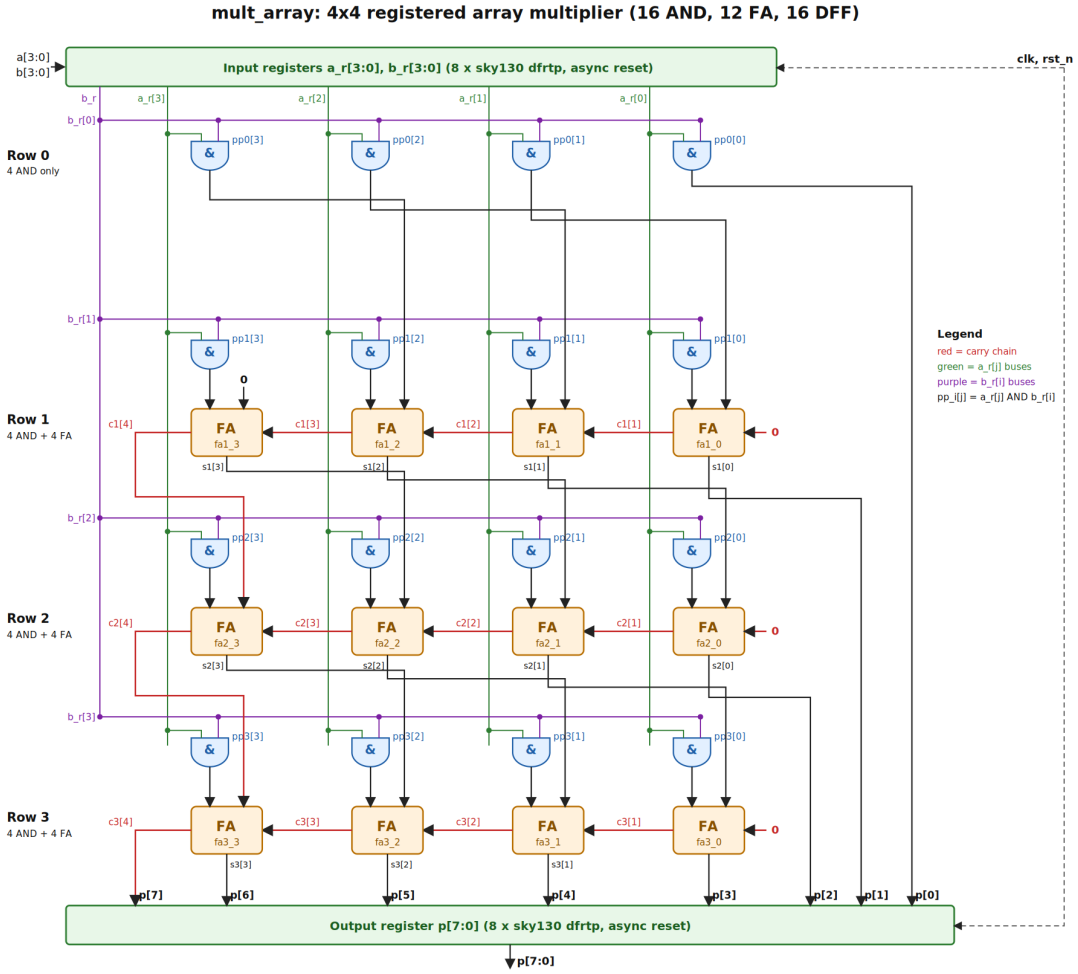


Fig. 1. Block diagram of `mult_array`: input registers (8 flip-flops), 16 AND gates (blue), 12 full adders in three ripple-carry rows (orange, carry chains in red) and the output register (8 flip-flops).

III. Design Flow and Tools

Table I lists the tools used. The SKY130 PDK [4] is used with the high-density standard-cell library `sky130_fd_sc_hd`; synthesis and STA use its typical-corner Liberty file `sky130_fd_sc_hd__tt_025C_1v80.lib`.

TABLE I
Tools and Versions

Tool	Version	Stage
Icarus Verilog [5]	14.0 (devel)	RTL / gate-level sim.
GTKWave	3.3.116	Waveforms
Yosys [6]	0.69	Synthesis
OpenSTA [7]	3.1.0	Timing analysis
LibreLane [8]	3.1.0.dev3	RTL-to-GDSII flow
OpenROAD [9]	(in LibreLane)	Place and route
Magic / Netgen [10]	8.3 / 1.5	DRC, extraction, LVS
KLayout [11]	0.30	DRC, layout views

IV. RTL Design and Verification

A. RTL

`rtl/full_adder.v` implements the full adder with two `assign` statements. `rtl/mult_array.v` (ports `clk`, `rst_n`, `a[3:0]`, `b[3:0]`, `p[7:0]`) contains the input registers, the 16 AND gates, the 12 explicitly instantiated `full_adders` wired exactly as in Section II-B, and the output register (Fig. 1). The $\star$ operator is not used.

B. Testbench

The self-checking testbench `tb/tb_mult.v`:

- generates a 10 ns clock and holds `rst_n` low for the first 2 cycles,
- applies all 256 combinations of `a` and `b` on the falling edge,
- waits two rising edges (the latency) and compares `p` with `a*b` 4 ns after the second edge ($\star$ is used only in the testbench),
- prints every mismatch and one summary line, and writes `mult.vcd`.

The RTL simulation result is:

```
PASS: 256 vectors checked, 0 errors
```

C. Error-Injection Test

To confirm that the testbench detects errors, `sum` in `full_adder.v` was temporarily changed to `a ^ b`:

```
MISMATCH at t=1069 ns: a=3 b=3 p=5 expected=9
...
FAIL: 256 vectors checked, 95 errors
```

The change was then undone. Only 95 of the 256 products fail, because the bug shows only when a carry-in of 1 actually reaches a full adder: products such as $x \times 0$, $x \times 1$ or $x \times 2$ never create an internal carry. Only an exhaustive testbench is guaranteed to expose such a fault. Fig. 2 shows both runs in the terminal.

D. Hand-Worked Example

The example uses Vansh Jaiswal's roll number 2024102013 (last digit 3): $A = 3 + 5 = 8$, $B = 7$.

```
          1 0 0 0      A = 8
        x 0 1 1 1      B = 7
---------------
          1 0 0 0      pp0 (b0 = 1)
        1 0 0 0        pp1 (b1 = 1), << 1
      1 0 0 0          pp2 (b2 = 1), << 2
    0 0 0 0            pp3 (b3 = 0), << 3
---------------
0 0 1 1 1 0 0 0        P = 56
```

Table II traces the same product through the array. Every row matches the simulated internal signals bit for bit.

TABLE II
8×7 TRACED THROUGH THE ARRAY

Row	Adds	s / $c[4]$	Bit	Passed down
0	$pp_0 = 1000$	–	$p_0 = 0$	$r_0 = 0100$
1	$1000 + 0100$	1100 / 0	$p_1 = 0$	$r_1 = 0110$
2	$1000 + 0110$	1110 / 0	$p_2 = 0$	$r_2 = 0111$
3	$0000 + 0111$	0111 / 0	$p_3 = 1$	$p_{7:4} = 0011$

$P = 0011\,1000_2 = 56 = 8 \times 7$.

E. Waveform

Fig. 3 shows the example in GTKWave: `a = 8`, `b = 7` are applied at 2730 ns, captured into `a_r`, `b_r` at edge 1 (2735 ns), and `p = 56` appears at edge 2 (2745 ns), a latency of 2 clock cycles (20 ns).

V. Logic Synthesis with Yosys

`synth/synth.tcl` runs the seven required Yosys steps: `read_verilog`, `synth -top mult_array -flatten`, `dfflibmap` and `abc` with the Liberty file, `opt_clean -purge`, `stat -liberty` (saved to `mult_array_stat.rpt`) and `write_verilog -noattr`. The results are in Table III.

TABLE III
PART B: SYNTHESIS RESULTS

Metric	Value
Total number of cells	63
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area	740.710 μm^2
Flip-flop area	$16 \times 25.024 = 400.384\,\mu$m^2
Flip-flop area (% of total)	54.05 %
Top 3 cell types	`dfrtp_1`×16, `nand2_1`×8, `a21oi_1`×6
Netlist simulation	`PASS: 256 vectors, 0 errors`

The other 33 cells are `xnor2_1`×4, `maj3_1`×3, `nand3_1`×3, `xor2_1`×3, `and2_0`×2, `clkinv_1`×2, `lpflow_isobufsrc_1`×2, `xnor3_1`×2 and 12 cell types used once.

A. Gate-Level Simulation

The netlist was simulated with the SKY130 cell models (`-DFUNCTIONAL -DUNIT_DELAY=#1`) and the same testbench, and passes all 256 vectors. The first gate-level run reported every product one vector late. The cause was in the testbench, not the design: with `UNIT_DELAY` the flip-flop models change `Q` 1 ns after the clock edge, exactly when the testbench originally sampled `p`. Sampling 4 ns after the edge removed this race; the RTL still passes and the error-injection test still fails.

VI. Static Timing Analysis with OpenSTA

`sta/sta.tcl` reads the typical-corner Liberty file and the netlist, defines a clock of period T on `clk`, sets 1.0 ns input and output delays, reports the worst setup slack and saves the critical-path report. At $T = 10$ ns the worst slack is 7.396 ns, so

$$T_{min} = T - WNS = 10 - 7.396 = 2.604 \text{ ns}, \quad (5)$$

$$F_{max} = 1000/T_{min} = 384.0 \text{ MHz}. \quad (6)$$

Rerunning at $T = 2.60$ ns (T_{min} rounded to 0.01 ns) gives a slack of -0.004 ns ≈ 0, and 2.61 ns gives $+0.006$ ns. With an ideal clock the path delay does not depend on T, so the slack is exactly $T - 2.604$ ns. Fig. 4 shows the gate-level simulation and both STA runs in the terminal.

TABLE IV
PART C: TIMING RESULTS (TT_025C_1V80)

Metric	Value
Worst slack at $T = 10$ ns	7.396 ns
T_{min}	2.604 ns ($\approx$ 2.60 ns)
F_{max}	384.0 MHz
Slack at $T = T_{min}$	-0.004 ns (≈ 0)
Critical path start point	`a_r[1]` register (CLK)
Critical path end point	`p[6]` register (D)
Cells on the critical path	9 (launch flip-flop + 8 gates)
Latency $= 2\,T_{min}$	5.208 ns
Throughput	384.0 million results/s

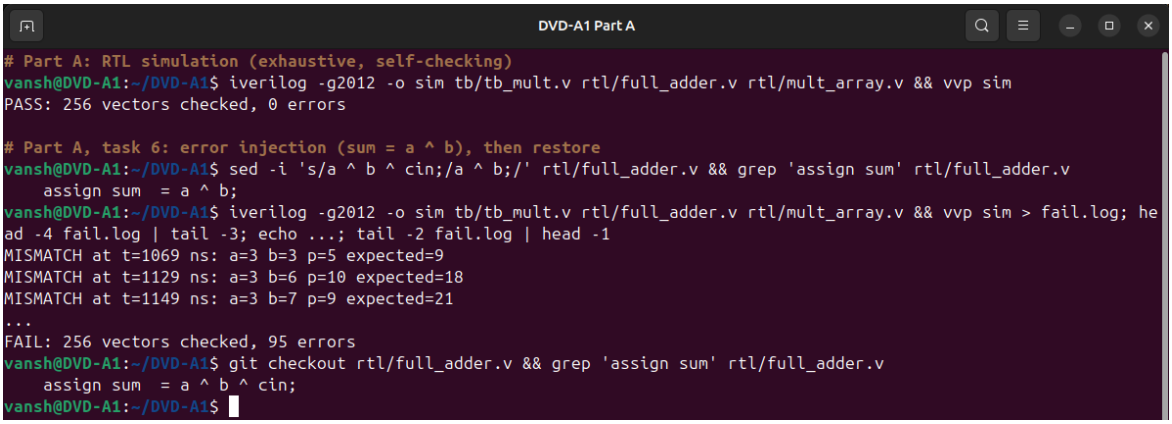


Fig. 2. Terminal output for Part A: the RTL simulation prints `PASS: 256 vectors checked, 0 errors`; with `sum` changed to `a ^ b` the same testbench prints `FAIL: 256 vectors checked, 95 errors`, after which the file is restored.

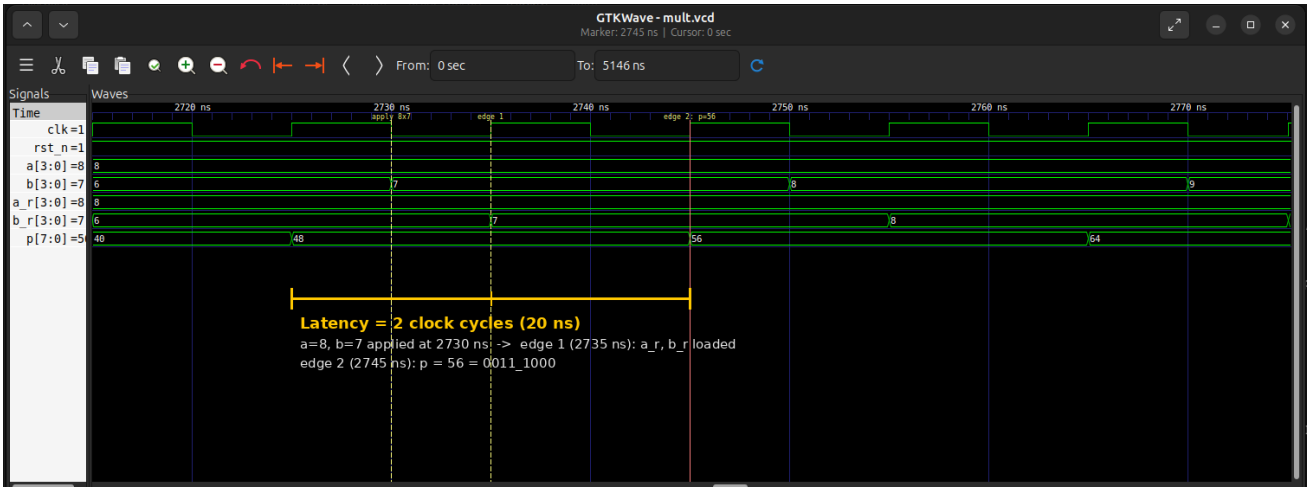


Fig. 3. GTKWave view of the example 8×7 with the 2-cycle latency marked: inputs applied at 2730 ns, input registers loaded at edge 1 (2735 ns), `p = 56` at edge 2 (2745 ns).

A. Critical Path

Table V lists the cells on the critical path. Each net was matched to an RTL signal by comparing its truth table over all 256 values of (`a_r`, `b_r`) with the equations of Section II.

TABLE V
CRITICAL PATH CELLS ($T = 10$ NS REPORT)

#	Cell	Delay (ns)	Arr. (ns)	Signal	Part of
0	dfrtp_1	0.398	0.398	a_r[1]	input reg.
1	clkinv_1	0.074	0.472	$\overline{a_r[1]}$	AND gates
2	o311ai_0	0.348	0.820	merged	AND + fa1_1
3	a21oi_1	0.172	0.992	$c_2[1]$	fa2_0 carry
4	maj3_1	0.381	1.373	$\overline{c_2[2]}$	fa2_1 carry
5	xnor2_1	0.160	1.534	$\overline{s_2[2]}$	fa2_2 sum
6	maj3_1	0.385	1.918	$\overline{c_3[2]}$	fa3_1 carry
7	maj3_1	0.377	2.295	$c_3[3]$	fa3_2 carry
8	xnor3_1	0.175	2.471	$s_3[3]$	fa3_3 sum

The path delay is 0.398 (clock-to-Q) + 2.073 (logic) + 0.134 (setup) = 2.605 ns $\approx T_{min}$. It crosses **7 full-adder stages**, fa1_1 $\rightarrow$ fa2_0 $\rightarrow$ fa2_1 $\rightarrow$ fa2_2 $\rightarrow$ fa3_1 $\rightarrow$ fa3_2 $\rightarrow$ fa3_3: along the row-2 carry chain, down through $s_2[2]$, and along the row-3 carry chain to the top sum bit. This agrees well with the estimate of about $2N = 8$ full-adder delays. Six of the eight gates compute a full-adder carry or sum directly, and each stage costs only one gate (≈ 0.30 ns) because the path needs either the carry or the sum of each adder, not both.

VII. Physical Design with LibreLane

A. Configuration and Runs

`pnr/config.json` sets:

- `DESIGN_NAME`: `mult_array`; `CLOCK_PORT`: `clk`,
- `VERILOG_FILES`: the two RTL files,
- `FP_SIZING`: `absolute`; `DIE_AREA`: $[0, 0, 150, 150]$, a fixed $150 \times 150\,\mu$m die,
- `CLOCK_PERIOD`: started at $1.2\,T_{min} = 3.125$ ns, rounded up to 3.5 ns.



Fig. 4. Terminal output for Parts B and C: the gate-level netlist simulation passes all 256 vectors; OpenSTA reports a worst slack of 7.40 ns (7.396 ns in the report) at $T = 10$ ns, from `_095_` (`a_r[1]`) to `_092_` (`p[6]`), and -0.00 ns at $T = T_{min} = 2.60$ ns.

The flow was run with `librelane config.json`, and the clock period was increased by 0.5 ns after every failing run (Table VI).

TABLE VI
LIBRELANE RUNS (SLACK IN NS)

Run	Period (ns)	Worst slack	Typ. slack	DRC	LVS	Result
1	3.5	-2.132	$+0.547$	0	0	fail
2	4.0	-1.632	$+1.047$	0	0	fail
3	4.5	-1.142	$+1.551$	0	0	fail
4	5.0	-0.642	$+2.052$	0	0	fail
5	5.5	-0.142	$+2.552$	0	0	fail
6	6.0	-0.064	$+2.943$	0	0	fail
7	**6.5**	$\mathbf{+0.085}$	$\mathbf{+3.293}$	**0**	**0**	**pass**

DRC and LVS were clean in every run; only setup timing failed, and only in the slow corner `max_ss_100C_1v60` (slow transistors, 100 °C, 1.60 V). LibreLane signs off in 9 corners and reports the worst as `setup__ws`; the typical corner of Part C passed from the first run. Runs 1–5 gained almost exactly 0.5 ns per step, but run 6 gained only 0.08 ns: LibreLane re-synthesises and re-sizes the design for each target period and uses smaller, slower gates when the clock is loose.

B. Results

TABLE VII
PART D: PHYSICAL-DESIGN RESULTS

Metric	Value
CLOCK_PERIOD used	6.5 ns
Number of runs needed	7
Core area	17 759.5 μm^2 (die 22 500 μm^2)
Cells after synthesis (Part B)	63
Cells after place and route	1 901
Kinds of cells added	decap, fill, tap, I/O, clock and fanout buffers (Table VIII)
Wirelength	2 405 μm
Setup slack (worst corner)	$+0.085$ ns
Post-route F_{max}	$1000/(6.5 - 0.085) = 155.9$ MHz
F_{max} from Part C	384.0 MHz
DRC errors	0 (Magic, KLayout, router)
LVS errors	0 (Netgen)

Further signoff results: hold slack $+0.124$ ns, no antenna or power-grid violations, identical Magic and KLayout GDS streams (XOR difference 0), and 7.0 % cell utilisation. The flow also reported 11 max-slew warnings, all in the slow corner and all on the asynchronous reset net (0.761 ns transition against a 0.75 ns limit); the typical corner has none and the flow completed successfully.

TABLE VIII
CELLS AFTER PLACE AND ROUTE

Kind	Cells	Count
Tap	`tapvpwrvgnd_1`	245
Decap	`decap_3/4/6/8/12`	1 296
Fill	`fill_1`, `fill_2`	270
Clock-tree buffers	`clkbuf_16`	3
I/O buffers	`buf_2` (9 in, 8 out)	17
Fanout buffers	`clkdlybuf4s50_1`	2
Design cells	`dfrtp_1`, logic, `inv_1`	68
Total		**1 901**

LibreLane runs its own synthesis script, which produced 68 design cells (16 flip-flops, 49 logic gates, 3 inverters) instead

of the 63 of Part B.

C. Layout

Fig. 5 shows the whole die and Fig. 6 a zoomed view of the logic.

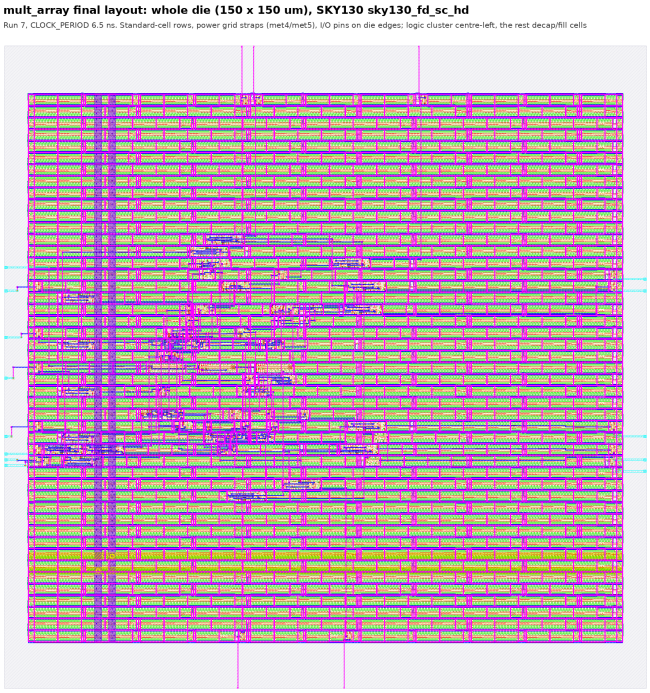


Fig. 5. Final layout, whole $150 \times 150\,\mu$m die (KLayout). Standard-cell rows under the power-grid straps and I/O pins on the die edges; the logic sits centre-left and the remaining space is decap and fill cells.

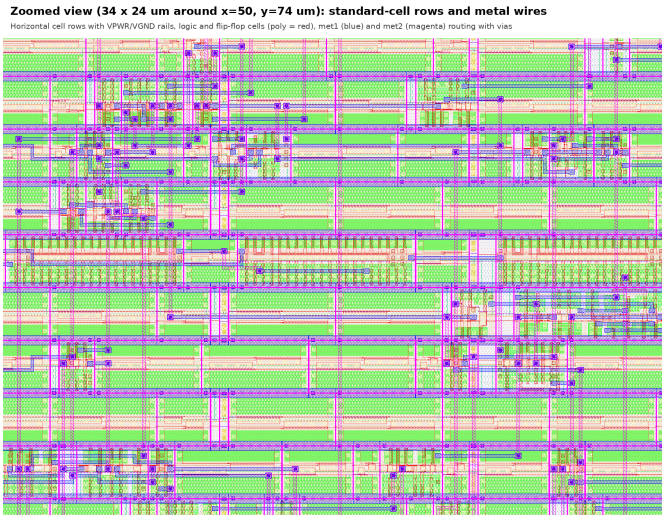


Fig. 6. Zoomed view ($34 \times 24\,\mu$m around the logic): standard-cell rows with VPWR/VGND rails, logic and flip-flop cells (polysilicon in red), metal-1 (blue) and metal-2 (magenta) wires with vias.

VIII. Answers to the Questions

A. Q1: Logic Cells after Synthesis

Synthesis produced $63 - 16 =$ **47 logic cells** ($340.3\,\mu\text{m}^2$) for 16 AND gates and 12 full adders (28 blocks). The numbers differ because ABC maps the flattened logic onto library cells rather than onto our modules. ABC does not use the multi-output full-adder cell (`fa_1`), so each adder becomes a sum gate (`xor`/`xnor`, sometimes `xor3`/`xnor3`) plus a carry gate (`maj3` or AOI/OAI/NAND logic), which raises the count. In the other direction, many AND gates are absorbed into complex gates (`o311ai`, `a21oi`, `nand2`), the column-0 adders have $c_{in} = 0$ and reduce to half adders, and the constant 0 fed into row 1 removes logic, which lowers it. Two inverters (`clkinv_1`) fix polarities. The resulting netlist is logically identical to the RTL, as the 256-vector gate-level simulation confirms.

B. Q2: Critical Path

The critical path runs from the **`a_r[1]` input register** to the **`p[6]` output register** and contains **9 cells**: the launch flip-flop and 8 gates (`clkinv`, `o311ai`, `a21oi`, `maj3`, `xnor2`, `maj3`, `maj3`, `xnor3`), with an arrival time of 2.471 ns and a 0.134 ns setup time at the typical corner. Matching every net to the RTL shows that it crosses **7 full-adder stages**, along the row-2 carry chain, through $s_2[2]$ into row 3 and along the row-3 carry chain to $s_3[3]$. This is close to the estimate of about $2N = 8$ full-adder delays, which is only an approximation of the longest chain. Each stage costs a single gate (≈ 0.30 ns) because the path uses only the carry or only the sum of each adder.

C. Q3: Flip-Flop Area

The flip-flops occupy $400.384/740.710 =$ **54.05 %** of the cell area, although they are only 16 of the 63 cells. A `dfrtp_1` is a master–slave flip-flop with asynchronous reset; at $25.02\,\mu\text{m}^2$ it is the largest cell in the design, about 6.7 times a `nand2_1` ($3.75\,\mu\text{m}^2$) and larger than any combinational cell used (the largest is `xor3_1` at $23.77\,\mu\text{m}^2$). The design is also very small: the array has only 28 AND/full-adder blocks, while registering both inputs and the output needs $4N = 16$ flip-flops. Registers grow linearly with N but the array grows as N^2; for $N = 8$ we would expect 32 flip-flops ($800.8\,\mu\text{m}^2$) against about 120 blocks ($\approx 1460\,\mu\text{m}^2$ at our $12.15\,\mu\text{m}^2$ per block), so the flip-flop share would fall to about 35 %.

D. Q4: Post-Route versus Pre-Layout F_{max}

The post-route F_{max} is **155.9 MHz** against **384.0 MHz** in Part C, because Part C was an idealised analysis:

1) *Corner:* Part C used only the typical corner, while LibreLane signs off in the slow corner (100 °C, 1.60 V), where the cells are about twice as slow. In the typical corner alone the routed design reaches $1000/(6.5 - 3.293) = 311.8$ MHz.
2) *Wires:* routed nets add resistance and capacitance, increasing gate delay and slew.

3) *Clock:* instead of an ideal clock, the clock passes through two levels of `clkbuf_16` (≈ 0.5 ns latency, with skew between flip-flops), and the flow adds 0.25 ns of clock uncertainty.
4) *Netlist:* LibreLane re-synthesises for the target period and uses smaller, slower gates when the clock is loose.

The flow added 1 833 cells (68 $\rightarrow$ 1 901). *Tap cells* (245) tie the n-well to VPWR and the substrate to VGND at regular intervals in every row to prevent latch-up. *Decap cells* (1 296) are capacitors between VPWR and VGND that supply charge locally when gates switch, reducing supply noise and IR drop. *Fill cells* (270) close the remaining gaps so that wells, implants and power rails are continuous along each row, as required by DRC. *Buffers*: three `clkbuf_16` form the clock tree that delivers `clk` to all 16 flip-flops with low skew, 17 `buf_2` I/O buffers isolate the 9 input and 8 output pins from the internal logic, and two fanout buffers split the high-fanout reset net.

E. Q5: DRC and LVS

Design rule checking (DRC) verifies that the layout geometry obeys the foundry's manufacturing rules: minimum widths and spacings of every layer, enclosures of contacts and vias, overlaps and densities. *Layout versus schematic* (LVS) extracts the transistors and their connections from the layout and checks that they form exactly the same circuit as the gate-level netlist, with the same devices, nets and pins. A DRC violation means the chip may not be manufacturable or will suffer shorts and opens that lower yield; an LVS mismatch means that a perfectly manufacturable chip would implement the wrong circuit. A layout can pass one check and fail the other, so both must pass before tape-out. Our final layout has **0 DRC errors** (Magic, KLayout and the router) and **0 LVS errors** (Netgen).

F. Q6: Scaling to 8 Bits

With $N = 8$ instead of $N = 4$:

$$\text{Full adders: } N(N-1) = 8 \times 7 = \mathbf{56} \quad (\text{vs. } 4 \times 3 = 12), \tag{7}$$

$$\text{AND gates: } N^2 = 64 \quad (\text{vs. } 16), \tag{8}$$

$$\text{Critical path: } \approx 2N = \mathbf{16} \text{ FA delays} \quad (\text{vs. } 8). \tag{9}$$

The number of full adders grows 4.7 times (quadratically), while the critical path only doubles (linearly), and the flip-flop count doubles to $4N = 32$. With our measured values (0.398 ns clock-to-Q, ≈ 0.296 ns per full-adder stage, 0.134 ns setup):

$$T_{min,8} \approx 0.398 + 16 \times 0.296 + 0.134 = 5.27 \text{ ns}, \tag{10}$$

i.e. $F_{max} \approx 190$ MHz at the typical corner, compared with 2.604 ns and 384.0 MHz for the 4-bit design.

IX. Conclusion

A registered 4-bit array multiplier was taken through a complete open-source RTL-to-GDSII flow in SKY130:

1) RTL design with 16 AND gates and 12 full adders, verified exhaustively (256/256) with a testbench that provably detects injected faults,
2) synthesis to 63 standard cells (740.71 μm^2) with a passing gate-level simulation,
3) static timing analysis giving $T_{min} = 2.604$ ns ($F_{max} = 384.0$ MHz) with a critical path through 7 full-adder stages, close to the $2N$ estimate,
4) place and route to a DRC- and LVS-clean 150×150 μm layout that meets multi-corner setup timing at 6.5 ns (155.9 MHz).

The drop from 384.0 MHz to 155.9 MHz shows how strongly signoff conditions that the pre-layout analysis ignores (the slow process corner, interconnect parasitics, the real clock tree and clock uncertainty) limit the achievable clock frequency, and why post-layout analysis is essential in practical VLSI design.

X. Individual Contributions

The work was divided equally between the two team members. By the assignment's own weighting, each member owns 50 of the 100 marks: Parts A and C ($30 + 20$) and Parts B and D ($20 + 30$). Each member also answered the questions that belong to their own parts:

- **Vansh Jaiswal (2024102013):** Part A (`full_adder.v`, `mult_array.v`, the self-checking testbench and its sampling fix, the error-injection test, the hand-worked example and the waveform figure) and Part C (`sta.tcl`, T_{min} and F_{max}, tracing the critical path through the full-adder stages); answers to Q2 and Q6; the GitHub repository and the README command log.
- **Saarthak Sabharwal (2023102055):** Part B (`synth.tcl`, synthesis, gate-level simulation and the cell/area table) and Part D (`config.json`, the LibreLane runs, the added-cell analysis, layout inspection in Magic and KLayout, DRC/LVS signoff and the layout figures); the block diagram; answers to Q1, Q3, Q4 and Q5; review of the pull request.

Both members wrote the report and cross-checked each other's results. All commits were pushed from Vansh's GitHub account, so the commit history shows a single author.

Acknowledgment

We thank the course instructor and the teaching assistants of Digital VLSI Design for their guidance, and the developers of the open-source EDA tools and the SkyWater PDK used in this work.

Use of AI Tools

An AI assistant was used to support parts of this work. The team guided it throughout, reviewed and verified every step and result, and takes full responsibility for the submitted design and report, in line with the responsible use of AI.

Source Code

All source files, scripts, generated reports, the final layout (GDSII) and the full commit history are available at https://github.com/VaNsH-IIIT/DVD-A1-array-multiplier.

References

[1] N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Pearson, ch. 11.

[2] J. Rabaey, A. Chandrakasan and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed. Pearson.

[3] "IIC-OSIC-TOOLS." [Online]. Available: https://github.com/iic-jku/IIC-OSIC-TOOLS

[4] "SkyWater SKY130 PDK." [Online]. Available: https://skywater-pdk.readthedocs.io

[5] "Icarus Verilog." [Online]. Available: https://steveicarus.github.io/iverilog

[6] "Yosys Open SYnthesis Suite." [Online]. Available: https://yosyshq.readthedocs.io/projects/yosys

[7] "OpenSTA." [Online]. Available: https://github.com/parallaxsw/OpenSTA

[8] "LibreLane." [Online]. Available: https://github.com/librelane/librelane

[9] "OpenROAD." [Online]. Available: https://openroad.readthedocs.io

[10] "Magic VLSI Layout Tool and Netgen." [Online]. Available: http://opencircuitdesign.com

[11] "KLayout." [Online]. Available: https://www.klayout.de